

NTFS Lab — Terraform File Reference

What goes in each .tf file, and why · Companion to the Complete Build SOP · Demarcus Miller

This is a file-by-file guide to the Terraform configuration itself — not the PowerShell orchestration (that's covered in the main SOP). Every code block below reflects the real structure and real values from the as-built lab, written as a reusable reference: copy the pattern, swap the values in Section 9 to stand up a different environment on top of it.

1. Why the config is split into multiple .tf files

Terraform doesn't care how many files a configuration is split across — it loads every `.tf` file in a directory and merges them into one configuration before doing anything. The split into `backend.tf`, `versions.tf`, `variables.tf`, `main.tf`, `keyvault.tf`, and `outputs.tf` is purely a human convention: each file groups one *kind* of concern, so anyone opening the project (including future-me) knows exactly where to look for a given piece of the puzzle instead of scrolling one 400-line file.

The convention used in this lab, and worth reusing on the next one:

- ``versions.tf`` — pins Terraform itself and every provider. Nothing else.
- ``backend.tf`` — where Terraform's state file lives. Nothing else.
- ``variables.tf`` — every input the configuration accepts, with type, description, and a default where one makes sense. No resources.
- ``main.tf`` — the provider block and the actual infrastructure: networking, NSG, VMs. The bulk of the build.
- ``keyvault.tf`` — pulled out of `main.tf` on purpose, since Key Vault + RBAC + secrets is its own concern with its own blast radius if misconfigured.
- ``outputs.tf`` — everything Terraform prints back out after `apply`, and everything other tooling (like `configure-lab.ps1`) reads back programmatically. No resources.

2. Quick Reference

File	What belongs in it
<code>versions.tf</code>	<code>terraform {}</code> block: required Terraform version, required providers and their version constraints.
<code>backend.tf</code>	<code>terraform { backend "azurerm" {} }</code> block pointing at the pre-created remote-state storage account.
<code>variables.tf</code>	Every <code>variable</code> declaration the config accepts — names, RDP source IP, VM sizes, the admin password (marked sensitive).
<code>main.tf</code>	<code>provider "azurerm"</code> , the resource group, VNet/subnet, NSG + rule, public IPs, NICs, and the three <code>azurerm_windows_virtual_machine</code> resources.
<code>keyvault.tf</code>	The Key Vault itself (RBAC-authorized), the role assignment granting access, and the secret holding the VM admin password.
<code>outputs.tf</code>	<code>output</code> blocks: VM public/private IPs, Key Vault name, resource group name — anything read back

File	What belongs in it
	after apply.
terraform.tfvars.example	A committed, safe-to-share template of variable values — no real secrets — that a new environment copies to terraform.tfvars and fills in.

3. versions.tf

Pins exactly which Terraform version and which provider versions this configuration was built and tested against, so a clone of this repo a year from now doesn't silently pull a newer provider that changes behavior underneath it.

versions.tf

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.100"
    }
    random = {
      source  = "hashicorp/random"
      version = "~> 3.6"
    }
    time = {
      source  = "hashicorp/time"
      version = "~> 0.11"
    }
  }
}
```

The *random* and *time* providers earn their place here even though most of this build's values are typed in by hand today: *random* is what generates the admin password once the Section-10-of-the-SOP automation lands, and *time* covers short waits Azure needs internally (e.g. letting a role assignment propagate before something tries to use it) that a plain resource dependency can't express.

4. backend.tf

Tells Terraform where to store its **state file** — the record of what it already created, used to diff against on every future `plan/apply`. Using a remote `azurerm` backend instead of a local `terraform.tfstate` file means the state survives a wiped laptop and never gets committed to git by accident.

backend.tf

```
terraform {
  backend "azurerm" {
    resource_group_name = "rg-tfstate-ntfslab"
    storage_account_name = "tfstatentfslab15577"
    container_name       = "tfstate"
    key                  = "ntfs-lab.tfstate"
  }
}
```

```
}
```

Chicken-and-egg problem worth remembering: Terraform can't create the storage account that its own backend depends on, so that resource group + storage account + blob container has to exist before `terraform init` is ever run — a one-time `az group create/az storage account create/az storage container create` sequence outside Terraform entirely. And whatever name you type for the storage account is only a request — Azure appended digits here (`tfstatentfslab` → `tfstatentfslab15577`) because the plain name was already taken globally. Always confirm the real assigned name with `az storage account list` before pasting it in above.

5. variables.tf

Every input the configuration accepts, and nothing else — no resources live here. Each `variable` gets a `description` (so `terraform plan` is self-documenting) and a `type`; only values that are genuinely safe to default (a VM size, a region) get a default. The admin password gets `sensitive = true` so Terraform never prints it in plan/apply output or logs.

variables.tf

```
variable "resource_group_name" {
  description = "Resource group that holds every resource in this lab"
  type        = string
  default     = "RG-FileServerLab"
}

variable "location" {
  description = "Azure region to deploy into"
  type        = string
  default     = "centralus"
}

variable "admin_username" {
  description = "Local admin username for every VM in the lab"
  type        = string
  default     = "labadmin"
}

variable "admin_password" {
  description = "Local admin password for every VM. Set via $env:TF_VAR_admin_password –
  never typed into a file."
  type        = string
  sensitive   = true
}

variable "domain_name" {
  description = "Active Directory domain promoted on DC01"
  type        = string
  default     = "lab.local"
}

variable "vnet_address_space" {
  type        = list(string)
  default     = ["10.0.0.0/16"]
}
```

```

variable "subnet_address_prefix" {
  type    = list(string)
  default = ["10.0.1.0/24"]
}

variable "dc01_private_ip" {
  description = "Static private IP reserved for DC01, so DNS never breaks after a reboot"
  type        = string
  default     = "10.0.1.4"
}

variable "allowed_rdp_source_ip" {
  description = "The one public IP the NSG allows through on port 3389 – no default, must be supplied"
  type        = string
}

variable "server_vm_size" {
  description = "DC01 / FS01 size. Must support Gen2 boot – see the SOP's known-issues log"
  type        = string
  default     = "Standard_D2_v4"
}

variable "client_vm_size" {
  type    = string
  default = "Standard_D2_v4"
}

variable "client_image_sku" {
  description = "Windows 11 Pro Marketplace SKU – verify with az vm image list-skus first, retired SKUs 404 without warning"
  type        = string
  default     = "win11-24h2-pro"
}

variable "key_vault_name" {
  description = "Must be globally unique across all of Azure, not just this subscription"
  type        = string
}

```

6. main.tf

The provider block and the bulk of the actual infrastructure: the resource group, the network (VNet + subnet), the NSG that locks the whole thing down to one inbound rule, and the three virtual machines. This file is intentionally the biggest one — everything in it either *is* the network boundary or sits inside it.

```

main.tf

provider "azurerm" {
  features {}
}

```

```

resource "azurerm_resource_group" "lab" {
  name      = var.resource_group_name
  location  = var.location
}

resource "azurerm_virtual_network" "lab" {
  name                = "vnet-fslab"
  address_space       = var.vnet_address_space
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
}

resource "azurerm_subnet" "lab" {
  name                = "snet-fslab"
  resource_group_name = azurerm_resource_group.lab.name
  virtual_network_name = azurerm_virtual_network.lab.name
  address_prefixes     = var.subnet_address_prefix
}

resource "azurerm_network_security_group" "lab" {
  name                = "nsg-fslab"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name

  security_rule {
    name                = "Allow-RDP-From-Me"
    priority            = 100
    direction           = "Inbound"
    access              = "Allow"
    protocol            = "Tcp"
    source_port_range    = "*"
    destination_port_range = "3389"
    source_address_prefix = var.allowed_rdp_source_ip
    destination_address_prefix = "*"
  }
  # Everything else is denied by the NSG's implicit DenyAllInBound rule.
}

resource "azurerm_subnet_network_security_group_association" "lab" {
  subnet_id                = azurerm_subnet.lab.id
  network_security_group_id = azurerm_network_security_group.lab.id
}

# --- Repeat this NIC + VM pattern for dc01, fs01, and client01 ---

resource "azurerm_public_ip" "dc01" {
  name                = "pip-dc01"
  location            = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
  allocation_method   = "Static"
  sku                 = "Standard"
}

resource "azurerm_network_interface" "dc01" {
  name                = "dc01-nic"

```

```

location          = azurerm_resource_group.lab.location
resource_group_name = azurerm_resource_group.lab.name

ip_configuration {
  name              = "internal"
  subnet_id         = azurerm_subnet.lab.id
  private_ip_address_allocation = "Static"
  private_ip_address = var.dc01_private_ip
  public_ip_address_id = azurerm_public_ip.dc01.id
}
}

resource "azurerm_windows_virtual_machine" "dc01" {
  name                = "DC01"
  resource_group_name = azurerm_resource_group.lab.name
  location            = azurerm_resource_group.lab.location
  size                = var.server_vm_size
  admin_username      = var.admin_username
  admin_password      = var.admin_password
  network_interface_ids = [azurerm_network_interface.dc01.id]

  os_disk {
    caching          = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "MicrosoftWindowsServer"
    offer     = "WindowsServer"
    sku       = "2022-datacenter-azure-edition"
    version   = "latest"
  }
}

# fs01: same NIC/VM pattern, dynamic private IP, same server_vm_size + image.
# client01: same pattern, client_vm_size, image sku = var.client_image_sku,
#           publisher "MicrosoftWindowsDesktop", offer "windows-11".

```

Domain promotion, domain join, share creation, NTFS ACLs, and GPOs are deliberately **not** Terraform's job here — Terraform's job stops at “three VMs exist, on the right network, reachable the right way.” Everything AD-specific happens afterward, driven by `configure-lab.ps1` through `az vm run-command` (see the main SOP, Section 2).

7. keyvault.tf

Split out of `main.tf` because Key Vault + access control is its own concern. Uses the **RBAC** authorization model (role assignments via Azure RBAC) rather than the older vault access-policy model — simpler to reason about since it's the same permission system as everything else in the subscription.

```

keyvault.tf

data "azurerm_client_config" "current" {}

resource "azurerm_key_vault" "lab" {
  name = var.key_vault_name

```

```

location          = azurerm_resource_group.lab.location
resource_group_name = azurerm_resource_group.lab.name
tenant_id         = data.azurerm_client_config.current.tenant_id
sku_name           = "standard"

enable_rbac_authorization = true
# NOTE: the provider docs reference a renamed `rbac_authorization_enabled`,
# but the schema doesn't actually accept it yet as of this build's provider
# version – keep using enable_rbac_authorization until it does.
}

resource "azurerm_role_assignment" "deployer_secrets_officer" {
  scope              = azurerm_key_vault.lab.id
  role_definition_name = "Key Vault Secrets Officer"
  principal_id       = data.azurerm_client_config.current.object_id
}

resource "time_sleep" "rbac_propagation" {
  depends_on      = [azurerm_role_assignment.deployer_secrets_officer]
  create_duration = "30s"
}

resource "azurerm_key_vault_secret" "vm_admin_password" {
  name      = "vm-admin-password"
  value     = var.admin_password
  key_vault_id = azurerm_key_vault.lab.id

  depends_on = [time_sleep.rbac_propagation]
}

```

This is exactly where automation item 1 from the main SOP plugs in later: `swap value = var.admin_password` for `value = random_password.vm_admin.result`, generated by a `random_password` resource instead of typed by hand — at that point `admin_password` disappears from `variables.tf` and the manual `$env:TF_VAR_admin_password` step goes away entirely. Not done yet, on purpose — see the SOP's automation roadmap.

8. outputs.tf

Everything printed back out after `terraform apply`, and everything other tooling reads back programmatically — `configure-lab.ps1` pulls VM names and the Key Vault name from here rather than having them hardcoded twice.

```

outputs.tf

output "resource_group_name" {
  value = azurerm_resource_group.lab.name
}

output "key_vault_name" {
  value = azurerm_key_vault.lab.name
}

output "dc01_public_ip" {
  value = azurerm_public_ip.dc01.ip_address
}

```

```

}

output "dc01_private_ip" {
  value = azurerm_network_interface.dc01.private_ip_address
}

output "fs01_public_ip" {
  value = azurerm_public_ip.fs01.ip_address
}

output "client01_public_ip" {
  value = azurerm_public_ip.client01.ip_address
}

```

Nothing sensitive gets an output — the admin password is retrieved from Key Vault at runtime by `configure-lab.ps1`, never printed by Terraform itself.

9. terraform.tfvars.example

The only `.tfvars` file that's safe to commit — placeholder values, no secrets. A new environment copies this to `terraform.tfvars` (which stays gitignored) and fills in real values.

```

terraform.tfvars.example
resource_group_name = "RG-FileServerLab"
location            = "centralus"
admin_username      = "labadmin"
allowed_rdp_source_ip = "203.0.113.10" # your real public IP, /32 implied
key_vault_name       = "kv-fslab-yourinitials"

# admin_password is deliberately NOT here.
# Set it per-session instead: $env:TF_VAR_admin_password = "..."

```

10. Variable Precedence — the lesson that cost real time

Terraform resolves a variable's value from several possible sources, highest priority first:

1. `-var / -var-file` flags passed directly on the command line
2. `*.auto.tfvars` files
3. `terraform.tfvars`
4. `TF_VAR_*` environment variables
5. the default in the `variable` block itself — lowest priority

This bit during the real build: a variable default was changed in `variables.tf` and `terraform plan` showed zero effect, because `terraform.tfvars` already set that value explicitly and always wins over a default. **Whenever a changed default doesn't seem to take, check `terraform.tfvars` first** — it's almost always the real source.

11. Adapting These Files for a New Build

What actually has to change when standing this config up again — for RBAC Lab 2, Azure Update Manager Lab 3, or any environment on top of this pattern:

- ``key_vault_name`` — must be globally unique across **all** of Azure, not just this subscription. Pick something new every time.
- ``backend.tf`` storage account / container — either reuse the existing remote-state backend with a new key, or bootstrap a fresh one; either way, verify the real assigned name with `az storage account list` before wiring it in.
- ``allowed_rdp_source_ip`` — your public IP changes across networks; update it or the NSG locks you out too.
- ``server_vm_size`` / ``client_vm_size`` — quota and live datacenter capacity are regional and can drift week to week. Re-check with `az vm list-usage` and `az vm list-skus` before trusting the existing defaults (see the main SOP's known-issues log for the exact failure modes: quota exceeded vs. `SkuNotAvailable` vs. hypervisor-generation mismatch — three different problems that look similar).
- ``client_image_sku`` — Marketplace SKUs get retired without warning. Re-verify with `az vm image list-skus` before every new build rather than trusting a value that worked last time.
- ``domain_name``, `resource names` — cosmetic, but keep them consistent across `main.tf`, the PowerShell scripts, and `outputs.tf` — they're referenced by string in a few places outside Terraform's own dependency graph.

12. Reflection

The Terraform side of this build was actually the more forgiving half — most of the real friction (Section 10's precedence bug, image SKUs disappearing, quota vs. capacity being two different failure classes) came from Azure's state at the moment of `apply`, not from anything wrong with the `.tf` files themselves. Splitting the config this way — one concern per file — made every one of those failures faster to isolate, because the fix was never “search a 400-line file,” it was “open `main.tf`” or “open `keyvault.tf`” and look exactly where the error pointed.

13. Postscript — Fixing a NIC Creation Race Condition

Discovered on the actual rebuild, after this reference was first written: `main.tf`'s `dc01/fs01/client01` NIC (Network Interface Card) block pattern in Section 6 has a hidden ordering problem. Terraform creates independent resources — ones with no attribute reference tying them together — in parallel. Nothing in the original NIC blocks referenced each other, so all three NICs could start creating at the same moment.

DC01's NIC asks for a specific, hardcoded address (`private_ip_address_allocation = "Static"`, `private_ip_address = var.dc01_private_ip`). FS01's and CLIENT01's NICs just ask for **Dynamic** — “give me whatever's free.” If a Dynamic NIC happens to finish first, Azure hands it the lowest available address in the subnet, which is exactly the address DC01's Static request is aiming for. DC01's NIC then fails outright with `PrivateIPAddressIsAllocated`, because that address is no longer free by the time its request lands.

The fix is an explicit `depends_on` on each Dynamic NIC, forcing it to wait until the Static NIC has actually been created — turning three parallel races into one guaranteed order: DC01's NIC claims its address first, every Dynamic NIC afterward gets whatever's left.

main.tf (corrected)

```
resource "azurerm_network_interface" "fs01" {
  name                = "fs01-nic"
  location             = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name

  ip_configuration {
    name                = "internal"
    subnet_id           = azurerm_subnet.lab.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id = azurerm_public_ip.fs01.id
  }

  depends_on = [azurerm_network_interface.dc01]
}

# client01's NIC gets the same depends_on line, same reason.
```

*Where this bit the second time: `depends_on` only changes the order Terraform creates *new* resources in — it does nothing to a resource that already exists in state. On the real rebuild, `client01-nic` had already been created (and had already grabbed the contested address) on an earlier apply, before this fix was saved. Adding `depends_on` afterward didn't move it or free the address — the fix only took effect once the already-created NIC was deleted (VM first — a NIC can't be deleted while attached to a running VM, or the delete fails with `NicInUse` — then the NIC) so Terraform recreated it fresh, this time waiting its turn like the config now says it should.*

Placement matters: `depends_on` is a top-level meta-argument on the resource block itself — a sibling to `name` and `location` — never nested inside `ip_configuration {}`. Putting it inside the wrong block is accepted by some editors without complaint but doesn't do anything; Terraform won't enforce the ordering.